Capítulo 3

Operadores sobre tablas y su implementación – parte 2

Tabla de contenidos

- Bloque 1: Ordenamiento
- Bloque 2: Ordenamiento externo
- Bloque 3: Ordenación selectiva
- Bloque 4: Operadores físicos de ordenación selectiva
- Bloque 5: Renombre
- Bloque 6: Remover duplicados
- Bloque 7: Operador físico para remover duplicados
- Bloque 8: Concatenación de tablas
- Bloque 9: implementación de concatenación de tablas
- Bloque 10: Resta
- Bloque 11: Implementación de resta
- Bloque 12: Intersección
- Bloque 13: Implementación de intersección

Bloque 1: Ordenamiento

- Especificamos **ordenamiento** usando la idea de insertion sort.
- La función *insert* inserta la tupla *t* en la tabla *r* en la posición que le corresponde: a izquierda las tuplas menores y a derecha las mayores.

$$\mathbf{insert}_{a_1,\ldots,a_N}\ \mathrm{t}\ \mathrm{r} = \sigma_{(a_1,\ldots,a_N)<\mathrm{t}[a_1,\ldots,a_N]}(\mathrm{r}) \mathbin{++} [\mathrm{t}] \mathbin{++} \sigma_{(a_1,\ldots,a_N)\geq\mathrm{t}[a_1,\ldots,a_N]}(\mathrm{r})$$

- El **ordenamiento ascendente** se define:
- $O_{a1,\ldots,aN}([]) = []$
- $O_{a1,\ldots,aN}(x{:}r) = \mathrm{insert}_{a1,\ldots,aN}(x, O_{a1,\ldots,aN}(r))$

Bloque 1: Ordenamiento

- El **ordenamiento descendente** se define como la reversa del ordenamiento ascendente.
- $O^{>}_{a1,\ldots,aN}(r) = \text{reverse}\ (O_{a1,\ldots,aN}(r))$
- Reverse [] = []
- Reverse (x : xs) = reverse xs ++ [x]
- Cuando queramos ordenar toda la tabla, omitiremos la lista de las columnas.

Operadores físicos de ordenamiento

- Ya saben ordenar tablas que caben en memoria principal.
- Estudiamos el caso de ordenamiento donde las tablas son mayores que la memoria principal.
  - Ordenar tablas que no caben en memoria se llama **ordenamiento externo**.
  - El algoritmo de ordenamiento externo más usado se llama **ordenamiento-combinación externo (external merge sort)**.

Bloque 2: Ordenamiento externo

- Sea *M* la cantidad de bloques en búfer de memoria principal disponibles para ordenación.

1. **Crear corridas ordenadas**

   i = 0

   repeat

      Leer *M* bloques de la tabla en memoria

      ordenar esos bloques en memoria

      Escribir los datos ordenados al archivo de ejecución R_i,

      i = i+1

   until el fin de la tabla

   - Sea N el valor final de i (cantidad total de corridas)

2. **Combinar las corridas**

Bloque 2: Ordenamiento externo

- Luego las corridas son combinadas. Suponemos que $N < M$.

  Leer un bloque de cada uno de los *N* archivos Ri en un bloque del búfer de memoria de corrida Ri.

  repeat

  Seleccionar la primera tupla en el orden de ordenamiento entre todos los bloques del búfer.

  Escribir la tupla al búfer de salida y borrarla del bloque del búfer donde estaba.

  if búfer de salida está lleno **then** escribirlo a disco, borrar contenido de búfer de salida.

  if búfer de corrida Ri está vacío y no fin de archivo Ri **then** leer el próximo bloque de la corrida Ri en el bloque de búfer de memoria correspondiente a esa corrida.

  until todos los bloques de búfer de las corridas estén vacíos.

Bloque 2: Ordenamiento externo

- Si $N \geq M$, varios ciclos de combinación son requeridos.
  - Se dividen las corridas en grupos contiguos de M-1 corridas.
  - Cada grupo de M-1 corridas es combinado (usando el algoritmo anterior), dando lugar a una nueva corrida.
  - Hacer los dos pasos anteriores constituye una pasada.
    - La misma reduce el número de corridas por un factor de M-1 y crea corridas más largas por el mismo factor.
  - Si el número de corridas resultantes de esa pasada es mayor a M,
    - se realiza otra pasada de la misma manera, pero con las corridas creadas en la primera pasada.
    - Observar que cada pasada reduce el número de secuencias en M-1.
  - Se repiten las pasadas hasta que número de corridas generadas sea menor que *M*.
  - Luego el último ciclo de combinación genera el resultado ordenado.

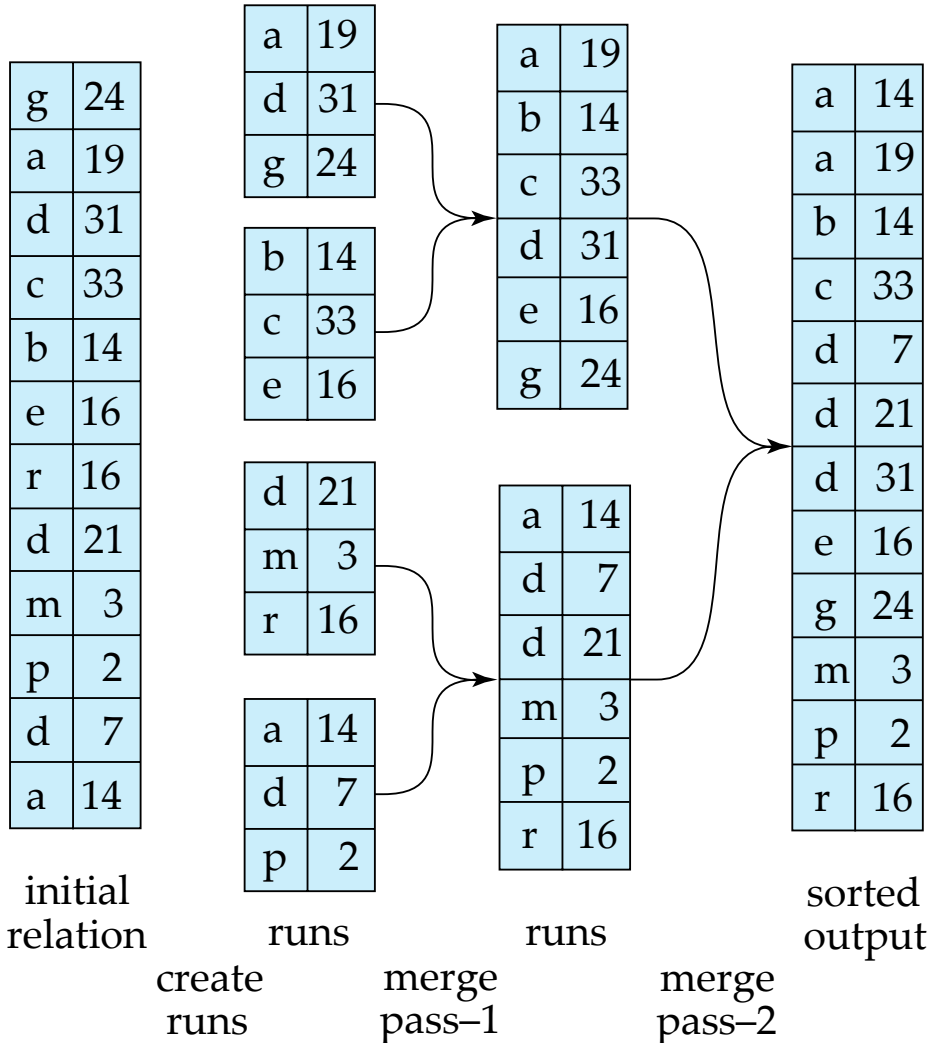
Bloque 2: Ordenamiento externo

Ejemplo: Se asume que:

- En memoria entran 3 bloques (M = 3).
- Se asume que bloques son de 1 tupla cada uno.

El algoritmo opera así:

- Primero se crean 4 corridas ordenadas.
- Luego se consideran grupos contiguos de M-1 = 2 corridas.
  - Se combinan grupos contiguos de 2 corridas. Se hacen 2 combinaciones.
  - Se obtienen así 2 corridas.
- Las dos corridas finales son combinadas dando el resultado ordenado final.



Bloque 2: Ordenamiento externo

- **Proposición:** Si hay *k* pasadas, entonces la cantidad de transferencias de bloques para ordenamiento externo es de:

  $b_r (2 k + 1)$

  - Donde b_r es cantidad de bloques de la tabla
- **Prueba**:
  - En crear corridas ordenadas hay que leer r: b_r transferencias de bloques.
  - Además una vez creadas esas corridas hay que escribirlas en disco. Esto es nuevamente b_r transferencias de bloques.
    - O sea tenemos 2* b_r transferencias de bloques.
  - Asumimos que hay k pasadas en total.
  - Para cada pasada se leen de disco y escriben a disco b_r bloques. Salvo en la última que asumimos que no se escribe a disco el resultado final.
  - Luego se terminan obteniendo $b_r (2 k + 1)$ transferencias de bloques.
- QED
- Se puede probar: k = 1 + ⌈log_{M−1}(⌈br/M⌉)⌉ pero no lo vamos a hacer.
- Una posibilidad es ponerle contador de pasadas al algoritmo.

Bloque 2: Ordenamiento externo

- **Proposición:** Sea *k* la cantidad de pasadas de mezcla. La cantidad de accesos a bloque es a lo sumo

$$2 \cdot \lceil b_r / M \rceil + b_r \cdot (2k - 1)$$

- **Prueba**:
  - **Generación de corridas:** se leen *M* bloques contiguos por seek y se escriben *M* bloques contiguos por seek. Hay $\lceil b_r / M \rceil$ corridas, así que son $2 \cdot \lceil b_r / M \rceil$ seeks.
  - **Cada pasada de merge:** por lo que ya probamos, se leen b_r bloques y se escriben b_r bloques. Cada transferencia necesita a lo sumo un seek, así que una pasada cuesta a lo sumo $2 \cdot b_r$ seeks.
  - **Última pasada:** no escribe el resultado final, así que cuesta a lo sumo br.
  - Sumando: $2 \cdot \lceil br/M \rceil + (k-1) \cdot 2 \cdot br + br = 2 \cdot \lceil br/M \rceil + br \cdot (2k-1)$.
- QED
- Hay resultados más sofisticados con prueba más complicada pero no lo vamos a hacer.

Bloque 3: Reunión selectiva

- Unimos las siguientes tablas mediante el campo *legajo*:

profe			
legajo	nombres	apellidos	sueldo
p1	"Benjamin"	"Pierce"	3000
p2	"Patricia"	"Selinger"	6000
p3	"Edgar F"	"Codd"	5500
p4	"Barbara"	"Liskov"	5600

curso		
id	legajo	nombre
c1	p2	"Optimización de Consultas"
c2	p3	"Fundamentos de BD"
c3	p2	"Análisis de Datos"
c4	p1	"Fundamentos del Software"
c5	p4	"Programación OO"

_						
legajo	nombres	apellidos	sueldo	id	legajo	nombre
p1	"Benjamin"	"Pierce"	3000	c4	p1	"Fundamentos del Software"
p2	"Patricia"	"Selinger"	6000	c1	p2	"Optimización de Consultas"
p2	"Patricia"	"Selinger"	6000	c3	p2	"Análisis de Datos"
p3	"Edgar F"	"Codd"	5500	c2	p3	"Fundamentos de BD"
p4	"Barbara"	"Liskov"	5600	c5	p4	"Programación OO"

- **La operación de combinar tablas por atributos en común es muy utilizada.** $\text{profe} \underset{\text{legajo}}{\bowtie} \underset{\text{legajo}}{\text{curso}}$

Bloque 3: Reunión selectiva

- Ahora pasamos a ver el esquema de la reunión selectiva.
- Sean dos tablas de esquema:

$$r(n_1 :: \tau_1, \ldots, n_N :: \tau_N) \text{ y } s(m_1 :: \tau'_1, \ldots, m_M :: \tau'_M)$$

- Entonces:

$$r \; {}_{a_1,\ldots,a_i}\!\bowtie_{b_1,\ldots,b_i} s :: (n_1 :: \tau_1, \ldots, n_N :: \tau_N, c_1, \ldots, c_{M-i})$$

$$\text{para } j \in [1, M-i],\ c_j \in \{m_i, \ldots, m_M\} \setminus \{b_1, \ldots, b_i\}.$$

Bloque 3: Reunión selectiva

- La reunión selectiva se puede definir usando los operadores vistos:

$$r \ {}_{a_1,\ldots,a_i}\bowtie_{b_1,\ldots,b_i} s = \Pi_{n_1,\ldots,n_N,c_1,\ldots,c_{M-i}}(\sigma_{a_1=b_1 \wedge \ldots \wedge a_i=b_i}(r \times s))$$

- Un tipo de reunión selectiva que se usa mucho es **reunión natural**: se aplica reunión selectiva a todos los atributos con el mismo nombre en las dos tablas.
- Sean $r(n_1 :: \tau_1, \ldots, n_N :: \tau_N)$ y $s(m_1 :: \tau'_1, \ldots, m_M :: \tau'_M)$, definimos la reunión natural de *r* y *s* como:

$$r \bowtie s = r \ {}_{a_1,\ldots,a_i}\bowtie_{a_1,\ldots,a_i} s$$

donde $\{a_1, \ldots, a_i\} = \{n_1, \ldots, n_N\} \cap \{m_1, \ldots, m_M\}$.

Bloque 3: Reunión selectiva
Reunión natural

- Relaciones *r*, *s*:

A	B	C	D
α	1	α	a
β	2	γ	a
γ	4	β	b
α	1	γ	a
δ	2	β	b

r

B	D	E
1	a	α
3	a	β
1	a	γ
2	b	δ
3	b	ε

s

- Los atributos en común son: B y D
- El esquema de la reunión natural de r y s: (A, B, C, D, E)

- $r \bowtie s$

A	B	C	D	E
α	1	α	a	α
α	1	α	a	γ
α	1	γ	a	α
α	1	γ	a	γ
δ	2	β	b	δ

Operadores físicos para reunión selectiva

```
Reunión_Selectiva(r, R, s, S, f, AL1, AL2)
  if not exist(f) then create(f, Rjoin) fi
  buffer_salida = vacío
  Br = leer_primer_bloque(r)
  while Br ≠ null do
    Bs = leer_primer_bloque(s)
    while Bs ≠ null do
      for tr in Br do
        for ts in Bs do
          if tr y ts coinciden para AL1, AL2
          then t' = tr • ts
                if no_hay_lugar(buffer_salida, t')
                then write(buffer_salida, f)
                      buffer_salida = vacío fi
                insertar(t', buffer_salida)
          fi   od  od
      Bs = leer_siguiente_bloque(s)  od
    Br = leer_siguiente_bloque(r)  od
  if buffer_salida ≠ vacío then write(buffer_salida, f) fi
```

- **Algoritmo de reunión selectiva de loop anidado de bloques**
  - **Estimación de peor caso:**
    - Asumimos bloques de r contiguos en disco y bloques de s contiguos en disco.
    - Cada bloque en la tabla interna *s* es leído una sola vez por cada bloque en la tabla externa r.
    - $b_r * b_s + b_r + b_f$ transferencias de bloque
    - $2 * b_r + 2 * b_f$ accesos a bloque porque para cada escritura de bloque en *f*, hay que volver después a acceder a un bloque de *s*.

Operadores físicos para reunión selectiva

```
Reunión_Selectiva2(r, R, s, S, f, M, AL1, AL2)
  if not exist(f) then create(f, R ⋈ S) fi
  buffer_salida = vacío
  for cada conjunto de (M-2) bloques Cr de r do
    Bs = leer_primer_bloque(s)
    while Bs ≠ null do
      for Br in Cr do
        for tr in Br do
          for ts in Bs do
            if tr y ts coinciden para AL1, AL2
            then t' = tr • ts
              if no_hay_lugar(buffer_salida, t')
              then write(buffer_salida, f)
                buffer_salida = vacío fi
              insertar(t', buffer_salida) fi od od od
    Bs = leer_siguiente_bloque(s) od od
  if buffer_salida ≠ vacío then write(buffer_salida, f) fi
```

- **Mejora del algoritmo anterior:**
  - Asumimos bloques de r contiguos y bloques de s contiguos.
  - M tamaño de memoria en bloques.
  - Usar M-2 bloques de disco para tabla externa, usar los restantes 2 bloques para tabla interna y salida.
  - Costo:
    - $\lceil b_r / (M-2) \rceil * b_s + b_r + b_f$ transferencias de bloques.
    - $2 \lceil b_r / (M-2) \rceil + 2 * b_f$ accesos a bloque

Operadores físicos para reunión selectiva

```
Reunión Selectiva_indexado(r, R, s, S, f, LA1, LA2)
// s tiene índice (B+) sobre los atributos de join; r es la externa
  if not exist(f) then create(f, Rjoin) fi
  buffer_salida = vacío
  Br = leer_primer_bloque(r)
  while Br ≠ null do
    for tr in Br do
      C = tr[LA1] // valor(es) de búsqueda
      // devuelve punteros a tuplas de s que matchean
      P = buscar_indice_join(C, LA2)
      for p in P do
        ts = leer_tupla(s, p) // vía leer_bloque + extraer tupla
        t' = tr • ts
        if no_hay_lugar(buffer_salida, t') then
          write(buffer_salida, f) buffer_salida = vacío fi
        insertar(t', buffer_salida) od od
    Br = leer_siguiente_bloque(r) od
  if buffer_salida ≠ vacío then write(buffer_salida, f) fi
```

- **Algoritmo de reunión selectiva de loop anidado indexado**
  - Asumir que hay un índice en atributos de la relación interna de la reunión.
  - Para cada tupla t_r en la tabla externa *r* usar el índice para buscar tuplas en *s* que satisfacen la condición de reunión con tupla t_r.
  - Si el índice es **primario**, devuelve un puntero al bloque de datos (caso clave candidata) o al primer bloque de un rango contiguo.
  - Si es **secundario con buckets**, devuelve la lista de punteros del bucket correspondiente a clave de búsqueda C.

Operadores físicos para reunión selectiva

```
buscar_indice_join(C, LA2)
  nodo = leer_bloque(indice, raiz)
  while nodo no es hoja do
    nodo = leer_bloque(indice,
    hijo de nodo que corresponde a C)
  od
  if existe entrada (C, pb) en nodo
  then  return punteros_del_bucket(pb)
  // o puntero único, según el tipo de  índice
  else return lista vacía fi
```

- Conviene cachear en memoria los bloques más altos del árbol (al menos la raíz) entre una búsqueda y otra, ya que se consultan repetidamente para distintos tr.
- b_r, n_r: bloques y tuplas de r (externa)
- b_f: bloques del resultado
- *c*: costo de **una** búsqueda por el índice (lectura del bloque de datos de *s* correspondiente a una tupla que matchea), según el tipo de índice:
  - **Índice primario sobre clave candidata**: c = hi + 1
  - **Índice primario sobre no-clave** (rango de tuplas contiguas que matchean): c = hi + *b*, donde *b* son los bloques que ocupan esas tuplas
  - **Índice secundario sobre clave candidata**: c = hi + 1
  - **Índice secundario sobre no-clave con buckets**: c = hi + k + n_match, con k bloques de bucket y n_match tuplas que matchean esa búsqueda particular

Operadores físicos para reunión selectiva

- **Costo del algoritmo de reunión selectiva de loop anidado indexado**
  - $b_r + n_r * c + b_f$ transferencias de bloques
  - n_r cantidad de registros de r.
  - Se lee r una vez (b_r),
  - y por cada una de las nr tuplas de *r* se hace una búsqueda en el índice de *s* (costo *c* cada una),
  - más la escritura del resultado.

- $b_r + n_r * c + 2 * b_f$ accesos.
- Cada búsqueda por índice es, en general, un conjunto de accesos no contiguos. Cada una aporta c seeks.
- Escritura de *f*: b_f seeks (peor caso) mas interrupciones al compartir cabezal con la lectura.
- Este algoritmo es mejor que loop anidado de bloques si $n_r * c < b_r * b_s$.
- Esto típicamente se cumple cuando *c* es chico (índice selectivo, pocas tuplas por valor) y n_r no es enorme.
- En el caso extremo, *c* se acerca a b_s y el algoritmo indexado pierde su ventaja.

Operadores físicos para reunión selectiva

Algoritmo de reunión por mezcla:

- Supongamos que hay un atributo de la reunión.
- Ordenar ambas tablas en el atributo de la reunión.
- Aplicar la reunión selectiva a las tablas ordenadas.
- Como puede haber varias tuplas con el mismo valor de del atributo en común en cada tabla, hay que **agrupar y hacer el producto cruzado** dentro de cada grupo con igual valor.
- **Nomenclatura**
  - Usaremos variables ps y pr para posición dentro de Bs (bloque actual de s) y posición dentro de Br (bloque actual de r) respectivamente.
  - Usaremos función tupla(B, p) que devuelve tupla de posición p dentro del bloque B.

```
Join_merge(r, R, s, S, JA, f)
  if not exist(f) then create(f, R ⋈ S) fi
  buffer_salida = vacío
  Br = leer_primer_bloque(r); pr = 0
  Bs = leer_primer_bloque(s); ps = 0
  tr = tupla(Br, pr);  ts = tupla(Bs, ps)
  while tr ≠ null and ts ≠ null do
    if tr[JA] < ts[JA]
      then (tr, Br, pr) = avanzar(r, Br, pr)
    else if tr[JA] > ts[JA]
      then (ts, Bs, ps) = avanzar(s, Bs, ps)
    else
      (tr, ts, Br, pr, Bs, ps, buffer_salida) =
      Join_grupo(tr[JA], r, Br,  pr, tr, s, Bs, ps, ts, f,
                              buffer_salida, JA)
    fi  od
  if buffer_salida ≠ vacío then write(buffer_salida, f) fi
  return f
```

Operadores físicos para reunión selectiva

Join grupo

- **Avance "en cremallera":** se avanza por la tabla cuyo valor de JA es menor, hasta igualar. Esto evita comparar todo contra todo.
- **Reproceso del grupo de s:** si hay múltiples tuplas de r con el mismo valor v, hay que *volver a recorrer el grupo de s con valor v* para cada una de ellas (producto cruzado).
  - Por eso se guarda (Bs_ini, ps_ini) y se "rebobina".
  - Si el grupo de s no entra en memoria, esto implica **releer desde disco** esos mismos bloques una vez por cada tupla de r con valor v — ahí está el costo extra cuando hay muchos duplicados.

```
Join_grupo(v, r, Br, pr, tr, s, Bs, ps, ts, f, BufSal, JA)
  (Bs_ini, ps_ini) = (Bs, ps)
  while tr ≠ null and tr[JA] = v do
    (Bs, ps) = (Bs_ini, ps_ini)
    ts = tupla(Bs, ps)
    while ts ≠ null and ts[JA] = v do
      t' = tr • ts
      if no_hay_lugar(BufSal, t') then
        write(BufSal, f)
        BufSal = vacío fi
      insertar(t', BufSal)
      (ts, Bs, ps) = avanzar(s, Bs, ps)
    od
    (tr, Br, pr) = avanzar(r, Br, pr)
  od
  return (tr, ts, Br, pr, Bs, ps, BufSal)
```

Operadores físicos para reunión selectiva

```
avanzar(tabla, B, p)
  p = p + 1
  if p >= cantidad_tuplas(B) then
    B = leer_siguiente_bloque(tabla)
    p = 0 fi
  if B = null then return (null, null, null)
  else return (tupla(B, p), B, p)
  fi
```

- **Análisis del algoritmo de reunión por mezcla**
- Si hay que ordenar las tablas hay que contar también esos ordenamientos.
- Además del costo de ordenar tenemos
- $b_r + \sum_{B\ bloque\ de\ s} nl(B) + b_f$ transferencias de bloques,
- donde *nl*(*B*) da la cantidad de veces que se tiene que leer el bloque B.
- Si todos los *nl*(*B*) = 1, entonces:
  - el término del medio de la fórmula es b_s.
  - Además tengo $b_r + b_s + b_f$ accesos a bloque.

Bloque 4: Operadores físicos para reunión selectiva

- **Algoritmo de reunión por mezcla híbrido:**
- **Suposición**: la primera tabla está ordenada y la segunda tabla está desordenada, pero tiene un índice secundario árbol B+ en los atributos de la reunión.
- **Algoritmo**:
  1. El algoritmo combina la tabla ordenada con las entradas hoja del índice secundario B+.
     - El archivo resultante contiene tuplas de la relación ordenada y direcciones para las tuplas de la relación no ordenada.
  2. Se ordena el resultado por las direcciones de las tuplas de la relación no ordenada,
  3. Permitiendo el retorno eficiente de las tuplas correspondientes en el orden de almacenamiento físico para terminar la reunión.
- **Costo**:
  - Primer paso: si T archivo generado, b_{ind} altura del árbol B+ mas cantidad de hojas en él, tenemos $b_r + b_T + b_{ind}$ transferencias de bloques.
  - Segundo paso: Costo de ordenar el archivo T.
  - Tercer paso: si T archivo ordenado del paso 2, tenemos $b_T + b_s + b_f$ transferencias de bloques.

Bloque 5: Renombre

- Sea la consulta: obtener el legajo de los profesores con sueldo anual > 60000.
- Tenemos de antes:

$$\text{sueldos_anuales} = \Pi_{\text{legajo},\ \text{sueldo}*13}(\text{profe})$$

- No podemos usar esta consulta y aplicarle selección.
- Para eso necesitamos darle nombre a la segunda columna de esta tabla.
- Esto se puede hacer con un operador de renombramiento más general que el anterior.

Bloque 5: Renombre

- Sea la tabla:

Sea $r(n_1 :: \tau_1, \ldots, n_N :: \tau_N)$.

$$\rho_{s(n'_1,\ldots,n'_N)}(r) :: s(n'_1 :: \tau_1, \ldots, n'_N :: \tau_N)$$

$$\rho_{s(n'_1,\ldots,n'_N)}(r) = r$$

- No lo ponemos, pero en el esquema de r algunos campos pueden no tener nombre.
- Entonces obtener el numero de legajo de profes con sueldo anual > 60000:
- $\sigma_{\text{sueldoAnual} > 600000}\ (\rho_{\text{profe2(nro_legajo, sueldAnual)}}\ (\Pi_{\text{nro_legajo, sueldo * 13}}\ (\text{profe})))$

Bloque 6: Remover duplicados

- La operación de remoción de duplicados se define recursivamente así:

1. *V* [ ] = [ ]
2. *V* (t:r) = if t∈r then *V* (r) else t: *V* (r)

- El predicado en la segunda ecuación se define del siguiente modo:
- $t \in [\,] = \text{false}$
- $t \in (u : r) = t == u \,||\, t \in r$

Bloque 7: Operador físico de remoción de duplicados

```
EliminarDuplicados(T_ordenado, R, f)
  if not exist(f) then create(f, R) fi
  buffer_salida = vacío
  B =  leer_primer_bloque(T_ordenado);  p = 0
  t_anterior = null // última tupla distinta vista
  t = tupla(B, p)
  while t ≠ null do
    if t_anterior = null or t ≠ t_anterior then
      if no_hay_lugar(buffer_salida, t) then
        write(buffer_salida, f)
        buffer_salida = vacío fi
      insertar(t, buffer_salida)
       t_anterior = t fi
   (t, B, p) = avanzar(T_ordenado, B, p) od
  if buffer_salida ≠ vacío then write(buffer_salida, f) fi
```

- **Algoritmo 1:**
  - **Fase 1**: Se ordena la tabla (según todos los atributos);
    - al hacerlo, todos los duplicados van a ser adyacentes entre sí.
  - **Fase 2**:
    - Y todos los duplicados menos uno, pueden ser eliminados (algoritmo a la izquierda)

```
avanzar(tabla, B, p)
  p = p + 1
  if p >= cantidad_tuplas(B)
  then B = leer_siguiente_bloque(tabla)
         p = 0 fi
  if B = null then return (null, null, null)
  else return (tupla(B, p), B, p) fi
```

Bloque 7: Operador físico de remoción de duplicados

- **Costo del algoritmo anterior**
  - Sea b_T los bloques de T ordenado y b_f los bloques del resultado sin duplicados.
  - Hay $b_T + b_f$ transferencias de bloques.
  - En el peor caso hay $b_T + b_f$ accesos a bloques.
- **Optimización del algoritmo anterior**:
  - en ordenación externa duplicados pueden ser eliminados durante generación de corridas,
  - así como en pasos de combinación intermedios.
  - El peor caso de costo para eliminación de duplicados es el mismo que el peor caso de costo para ordenar una tabla.

Bloque 8: Concatenación de tablas

- ¿Qué operación de tablas corresponde a la unión de conjuntos?
- Una posibilidad es la **concatenación de tablas**.
- Es como la concatenación de listas solo que ahora el resultado tiene un esquema.
- Los esquemas de los operandos deben ser **compatibles (**i.e. igual cantidad de columnas y mismos tipos en las mismas posiciones**)**.

$$\text{Sean } r(n_1 :: \tau_1, \ldots, n_N :: \tau_N) \text{ y } s(m_1 :: \tau_1, \ldots, m_M :: \tau_N)$$

$$r{+}{+}s :: (n_1 :: \tau_1, \ldots, n_N :: \tau_N)$$

- [] ++ s = s
- (t:r)++ s = t: (r++s)

Bloque 9: Implementación de concatenación

- Concatenación requiere leer ambas tablas: primero la de la izquierda y luego la de la derecha.
  - A medida que se lee una tabla se genera el resultado.
- **Análisis de costo**:
  - En el peor caso para computar r++s vamos a necesitar:
    - $b_r + b_s$ transferencias de bloques y
    - $b_r + b_s$ accesos a bloques (cuando todos los bloques de *r* y *s* no están contiguos)
- **Nota**: Si el resultado se guarda a disco: además se escriben $b_r + b_s$ bloques en disco.

Bloque 10: Resta

Los esquemas de las tablas deben ser compatibles

$$r(n_1 :: \tau_1, \ldots, n_N :: \tau_N) \text{ y } s(m_1 :: \tau_1, \ldots, m_M :: \tau_N)$$

El esquema de la resta es el del primer operando.

$$r \setminus s :: (n_1 :: \tau_1, \ldots, n_N :: \tau_N)$$

$$r \setminus s = \sigma_{(\backslash \mathsf{t} \to \mathsf{t} \notin \mathsf{s})}(r)$$

Bloque 11: Implementación de Resta

- **Algoritmo**:
  - Para calcular la resta de dos tablas, podemos primero **ordenarlas** a ambas por la **clave primaria** que asumimos que es la misma en ambas tablas.
  - Luego podemos escanear una vez cada tabla ordenada para producir el resultado.

```
Resta(r_ordenado, R, s_ordenado, S, AK, f)
  // AK: atributos de clave primaria de r
  if not exist(f) then create(f, R) fi
  buffer_salida = vacío
  Br = leer_primer_bloque(r_ordenado); pr = 0
  Bs = leer_primer_bloque(s_ordenado); ps = 0
  tr = tupla(Br, pr) ts = tupla(Bs, ps)
  while tr ≠ null do
    if ts = null or tr[AK] < ts[AK] then
      if no_hay_lugar(buffer_salida, tr)  then
        write(buffer_salida, f)
        buffer_salida = vacío fi
      insertar(tr, buffer_salida)
      (tr, Br, pr) = avanzar(r_ordenado, Br, pr)
    else if tr[AK] > ts[AK] then
      (ts, Bs, ps) = avanzar(s_ordenado, Bs, ps)
    else // tr[AK] = ts[AK]: tr está en s, se descarta
      v = tr[AK]
      (tr, Br, pr) = avanzar(r_ordenado, Br, pr)
      while ts ≠ null and ts[AK] = v do
        (ts, Bs, ps) = avanzar(s_ordenado, Bs, ps)
      od fi od
  if buffer_salida ≠ vacío then write(buffer_salida, f) fi
```

Bloque 11: Implementación de Resta

```
avanzar(tabla, B, p)
  p = p + 1
  if p >= cantidad_tuplas(B) then
    B = leer_siguiente_bloque(tabla)
    p = 0
  fi
  if B = null then return (null, null, null)
  else return (tupla(B, p), B, p) fi
```

- **Costo del algoritmo anterior**:
  - Suponiendo las tablas r, s ordenadas de ese modo.
  - Si un solo bloque en búfer se usa por tabla:
    - $b_r + b_s + b_f$ transferencias de bloques
    - Un cota superior para la cantidad de accesos a bloque es de: $b_r + b_s + b_f$

Bloque 12: Intersección

- Los esquemas de las tablas deben ser compatibles

$$r(n_1 :: \tau_1, \ldots, n_N :: \tau_N) \text{ y } s(m_1 :: \tau_1, \ldots, m_M :: \tau_N)$$

- El esquema de la intersección es el del primer operando.

$$r \cap s :: (n_1 :: \tau_1, \ldots, n_N :: \tau_N)$$

$$r \cap s = \sigma_{(\mathsf{t} \to \mathsf{t} \in \mathsf{s})}(r)$$

Bloque 13: Implementación de intersección

- **Algoritmo**:
  - Para calcular la intersección de dos tablas, podemos primero **ordenarlas** a ambas por la clave primaria, que asumimos que es la misma en ambas tablas.
  - Luego podemos escanear una vez cada tabla ordenada para producir el resultado.
- **Costo**: Suponiendo las tablas r, s ordenadas de ese modo, r ∩ s requiere:
  - Si un solo bloque en búfer se usa por tabla:
    - $b_r + b_s + b_f$ transferencias de bloques
    - Cota de accesos a bloque: $b_r + b_s + b_f$